

SECURING THE SDLC AND SOFTWARE DEVELOPMENT ECOSYSTEM

Objectives

- Describe development methodologies and maturity models
- Provide an overview of DevOps operation, maintenance and change management, and integrated product teams
- Compare programming languages, libraries, integrated development environments, toolsets, and runtime environments
- Describe continuous integration (CI)/ continuous delivery (CD) and Agile life cycles
- Provide an overview of software configuration management (SCM), code repositories, and application security testing methods

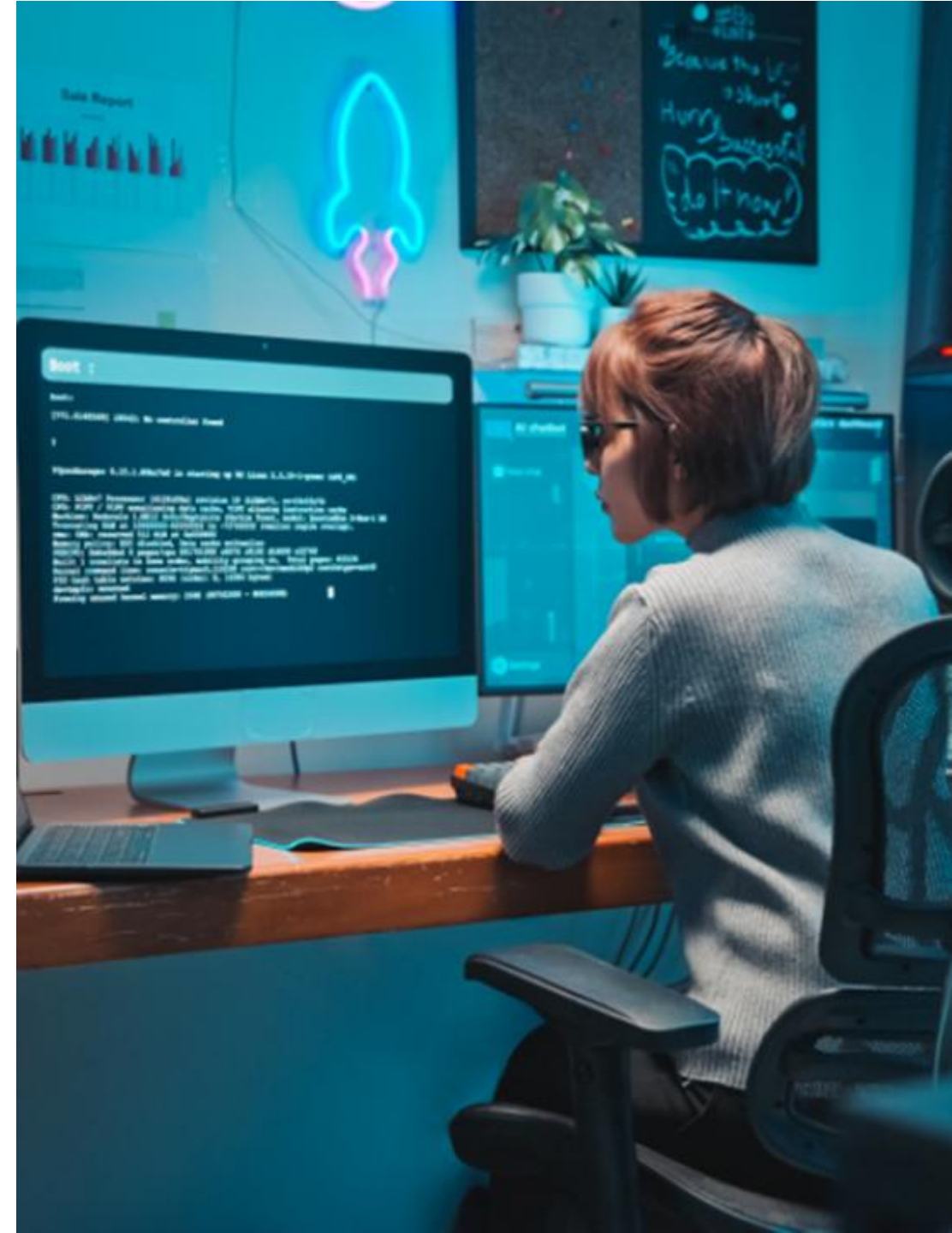
SECURE BY DESIGN

- The custom or outsourced program or application is developed with security integrated into the entire software development life cycle (SDLC)
- Security by design ensures apps are constructed with security in mind from the outset using the following principles:
 - Least privilege - limits access to only what is necessary
 - Segregation of duties - partitions responsibilities to prevent abuse of the application
 - Defense in depth - uses several layers of security
 - Failing securely - handles errors carefully
 - Open design - avoids security through obscurity

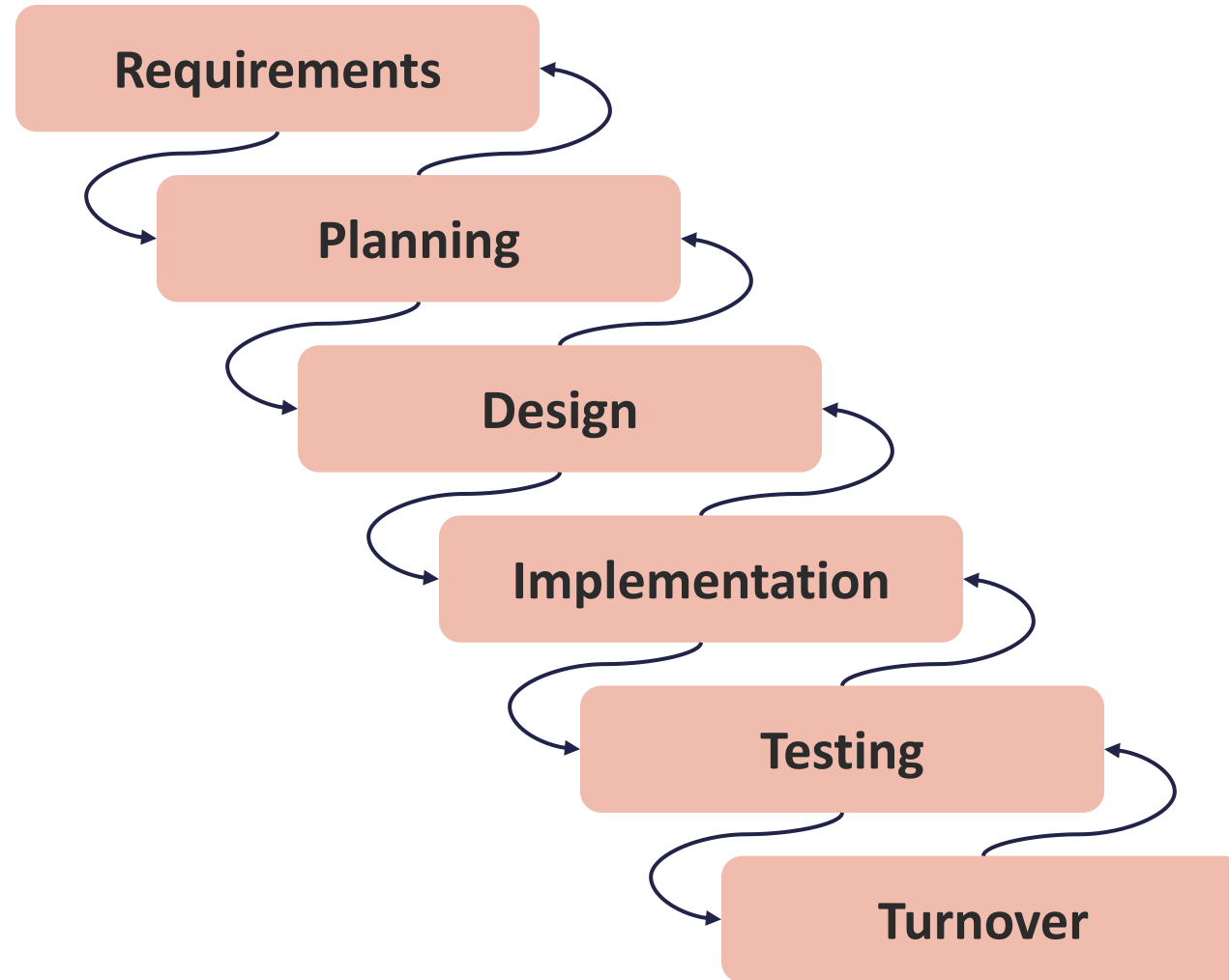


SECURE BY DEFAULT

- This application design consideration assumes the application is natively secure without any modifications or additional controls:
 - Example 1: a server application has certain possible unsecure functions, but they are disabled by default at deployment based on Infrastructure as Code (IaC)
 - Example 2: building an application solution on tested and proven containers and microservices linked by secure application programming interfaces (APIs)



DEVELOPMENT METHODOLOGIES: WATERFALL

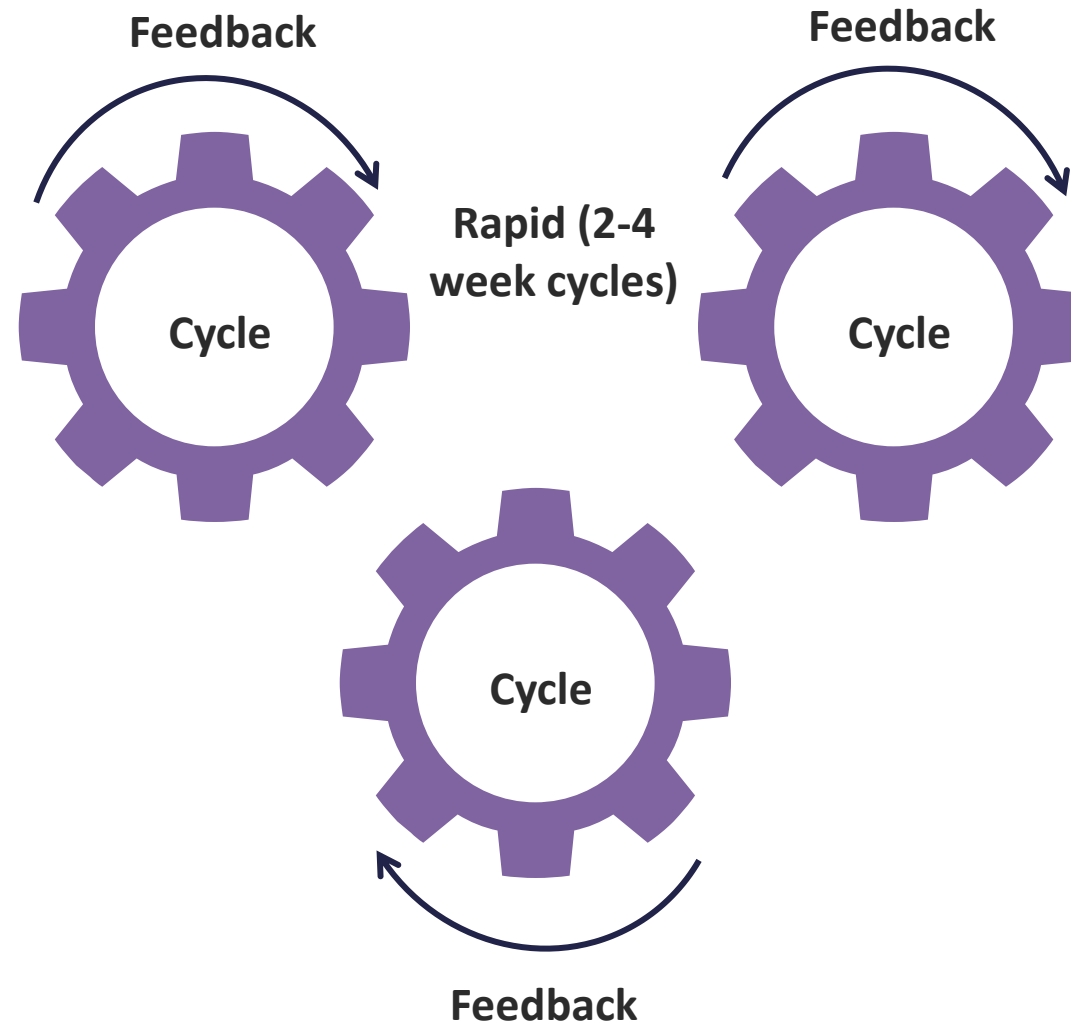


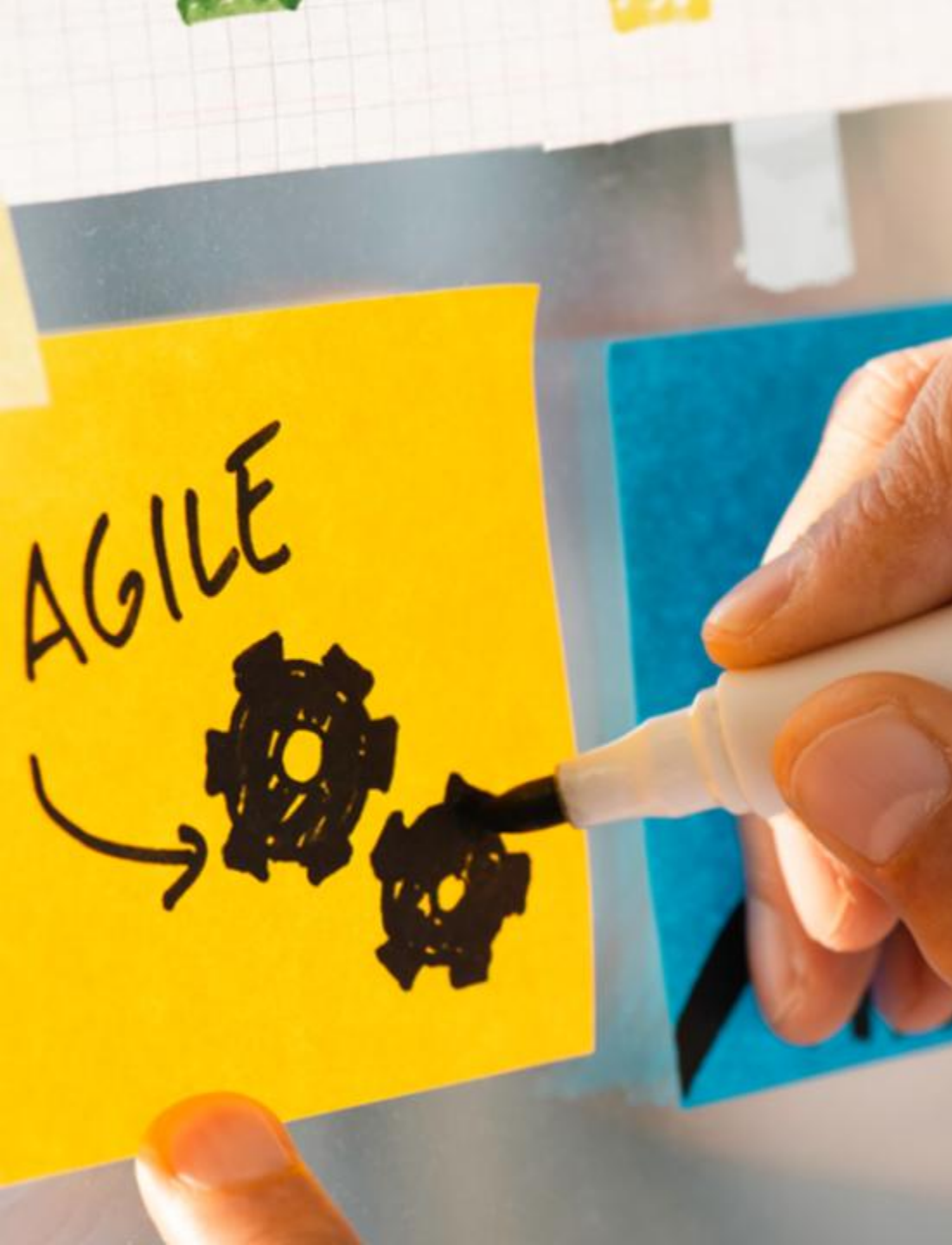


AGILE DEVELOPMENT

- Agile is an evolutionary approach that is measured in weeks and involves collaboration between cross-functional teams
- It is widely used and excellent for smaller and rapidly developed projects
- Agile is flexible, adaptable, not predictable, and testing is done during development
- Estimates (i.e., budget, schedule, etc.) get more realistic as work progresses, because important issues are discovered earlier

DEVELOPMENT METHODOLOGIES: AGILE





SCALED AGILE FRAMEWORK

- The Scaled Agile Framework (SAFe) is a collection of structural and workflow outlines for deploying Agile practices at scale
- It is a free and open online knowledge base that offers organized direction on roles and responsibilities
- The framework shows how to plan and manage the workflow
- The goal is to meet the challenges of scaling Agile development in large, complex, distributed, or compliant heavy environments

SCALED AGILE FRAMEWORK

- There are four pillars of SAFe:
 - **Alignment** assures that all participants are working on the same goals
 - **Built-in quality** focuses on quality from the outset
 - **Transparency** makes information observable and available
 - **Program execution** delivers value through effective implementation





DevSecOps

- DevSecOps is the integration of security practices directly into the DevOps pipeline, ensuring that security is not an afterthought but a foundational component of the software development lifecycle
- This approach **shifts security “left”** by embedding it early in planning, coding, and testing phases so vulnerabilities can be identified and mitigated before deployment
- It emphasizes collaboration among development, operations, and security teams, fostering shared responsibility for protecting systems and data

DevSecOps

- Automation plays a key role, with tools for continuous integration, code analysis, and compliance checks helping maintain security without slowing down delivery
- DevSecOps aligns with principles of secure system development, risk management, and continuous monitoring
- It supports the broader goals of confidentiality, integrity, and availability by ensuring that security controls are baked into infrastructure as code, container orchestration, and CI/CD workflows





CAPABILITY MATURITY MODEL (CMM)

- The Capability Maturity Model is a methodology and initiative used to advance and enhance an organization's system and software development processes
- The model consists of a five-level evolutionary path of progressively prepared and systematically more mature processes
- It is like ISO 9001 standards that specify an effective quality system for manufacturing and service industries

CMM LEVEL 1

- At the **initial (chaotic) level 1**, processes are disorganized, even chaotic
- Success most often depends on individual heroics and is not considered to be repeatable
- Processes are not sufficiently defined or documented for them to be replicated
- Decision-making is a free-for-all and based on intuition and existing experience
- Decision-making authorities are poorly defined, with no key risk indicators (KRIs), key performance indicators (KPIs), critical success factors (CSFs), or real meaningful metrics





CMM LEVEL 2

- At the **repeatable (implicit) level 2**, fundamental project management is established, and successes could be repeated, as the obligatory processes have been established, somewhat defined, and documented
- Decision-making is based on rote adherence to poorly-aligned standards and practices, and data provided to decision-makers is often superficial, not codified, and cannot hold up to scrutiny
- Roles and responsibilities are unclear, and it is common for people to make decisions without authorization
- Established KRIs and metrics, if any, are questionable as risk terminology, taxonomy, and policy is superficial or non-existent

CMM LEVEL 3

- At the **defined (early explicit) level 3**, the enterprise has developed its own standard software process through greater attention to documentation, standardization, and integration
- Standardized terminology exists and assessments are more up-to-date and better supported
- Visibility is improved as more robust and defensible analysis exists
- Well established standards, security teams, and steering committees exist
- Risk registers, KPIs, KRIs, CSFs and meaningful metrics and calibrated and more precise





CMM LEVEL 4

- At the **managed (mature explicit) level 4**, the processes are controlled and can quickly be adjusted and adapted for development projects, security initiatives, or other endeavors without measurable loss of quality
- Quality is visible and risk registers and assessments are up-to-date
- Data is actively used, and risk treatment/handling is adapted accurately based on more quantitative analysis
- Indicators and metrics are well-defined and tested
- Methods for assurance and certification are established if pertinent

CMM LEVEL 5

- At the **optimized (purely explicit) level 5**, the process management has attained everything at level 4, including deliberate process optimization and continual improvement
- Level 5 is rarely achieved, but is still possible with proper leadership and resources
- For example, ITIL 4 mastery and maximum software development proficiency would be demonstrated in this organization

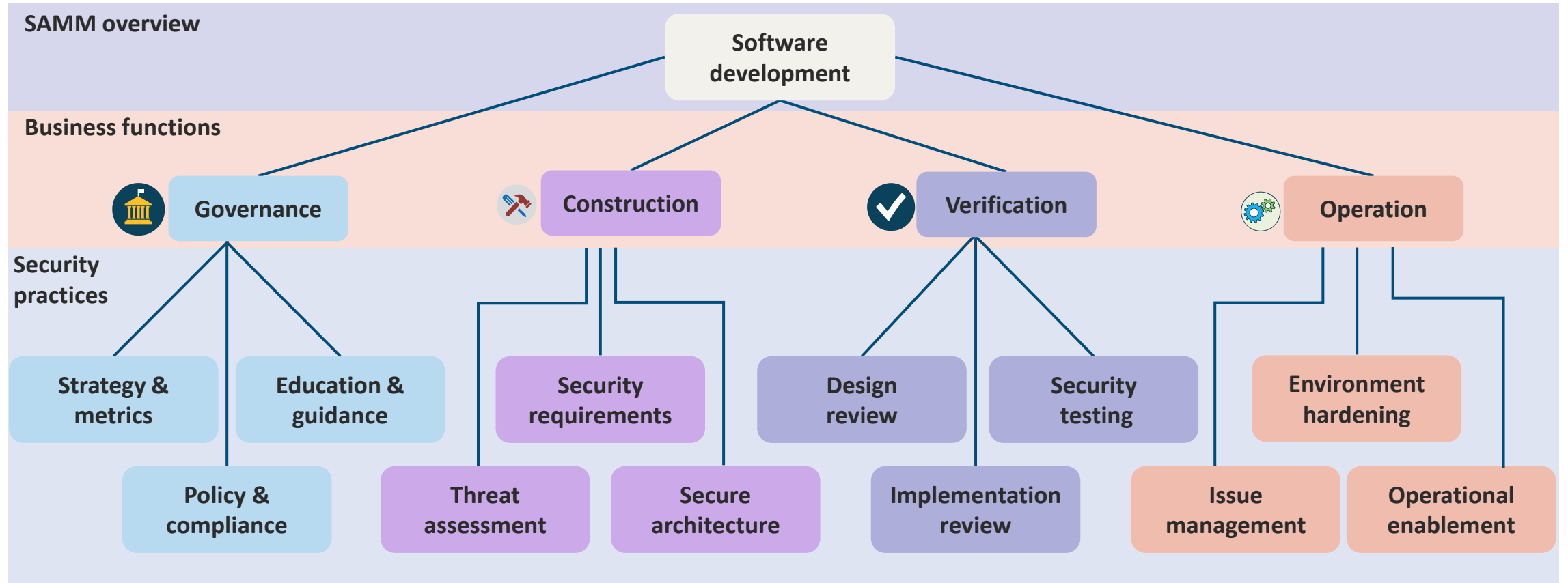


A photograph of a person's hands typing on a keyboard at a wooden desk. A large computer monitor displays a code editor with syntax-highlighted code. A white coffee cup sits on the desk. The background is a blurred office environment with a staircase.

SOFTWARE ASSURANCE MATURITY MODEL (SAMM)

- SAMM is an open framework from OWASP to assist organizations in developing and deploying a secure software delivery strategy that is focused on the detailed risks facing the enterprise
- The resources offered by SAMM will assist in:
 - Appraising the organization's current software security initiatives
 - Constructing a well-adjusted software security assurance program using established iterative processes
 - Establishing tangible continual improvement methodologies to a software security assurance program
 - Defining and gauging security-related tasks throughout the enterprise

SAMM FRAMEWORK



"Software Assurance Maturity Model." OWASP.org. Accessed June 8, 2021. https://owasp.org/www-pdf-archive/SAMM_Core_V1-5_FINAL.pdf.

DevOps OPERATIONS

- DevOps operations are the tasks and processes that empower application development teams to successfully plan, construct, distribute, and workflow software products, applications, and systems
- DevOps operations include processes like version control, Agile and CI/CD development, Infrastructure as Code, configuration management, and constant monitoring and visibility
- Operations target improved collaboration, quality, speed, and dependability of software deployment

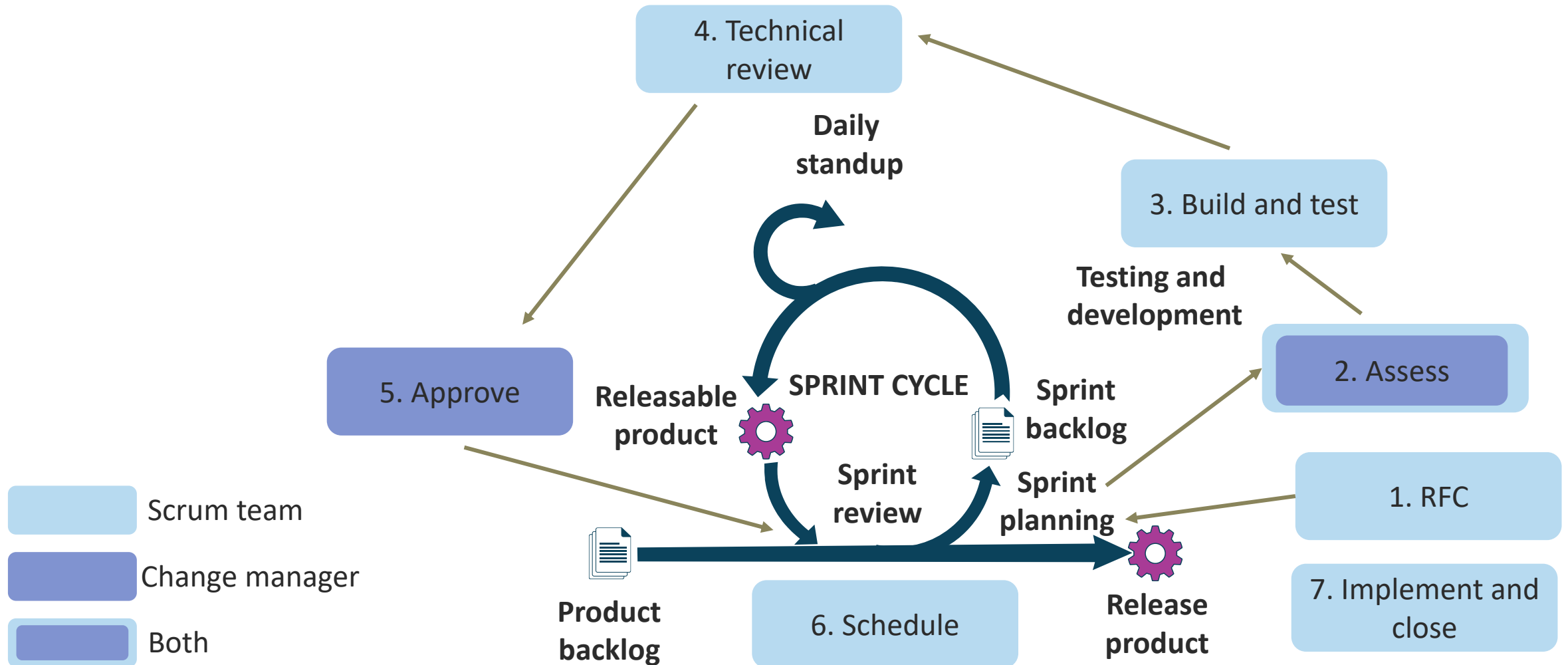


DevOps MAINTENANCE

- DevOps teams strive to certify application reliability, high availability, and performance
- Maintenance also involves reinforcing security and governance
- To maintain DevOps, developers will:
 - Evaluate and improve code frequently
 - Plan for failure in a proactive way
 - Gain visibility into software performance
 - Fix bugs and errors
 - Track user feedback and experience (CX/UX) and address them in near real-time
 - Confirm that programmers are accountable for assuring that code works in production



DevOps Change Management (ITIL 4)





INTEGRATED PRODUCT TEAMS (IPTs)

- An IPT is defined as a multidisciplinary team of people who are collectively responsible for delivering a well-defined product or service solution
- The Department of Defense (DoD) has adopted IPTs as their preferred approach for systems and software acquisition
- IPTs are maintained by different subject matter experts and can be leveraged throughout the entire development life cycle

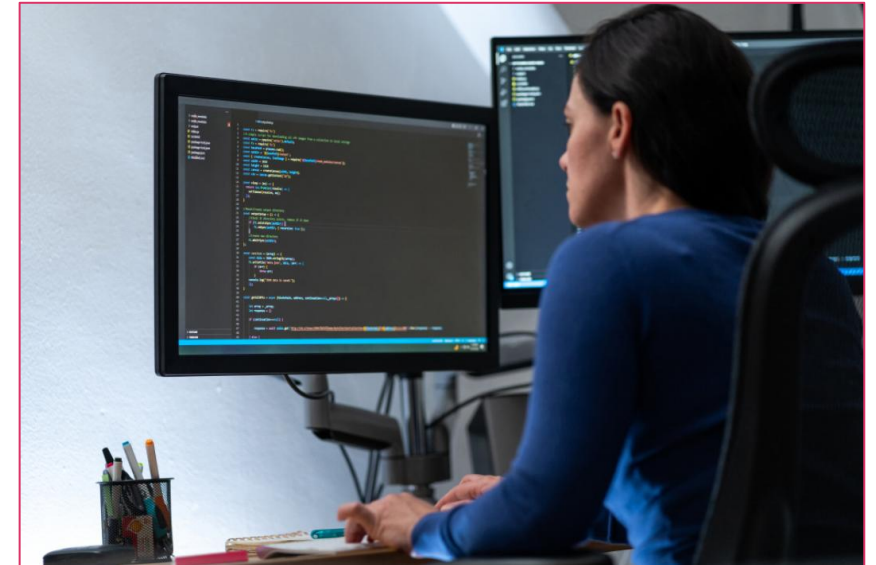
INTEGRATED PRODUCT TEAMS (IPTs)

- These groups of key individuals represent varying ranges of expertise with the common goal of delivering the best product or service
- This cross-functional expert judgment also supports product acquisition activities and the development of system and software solutions

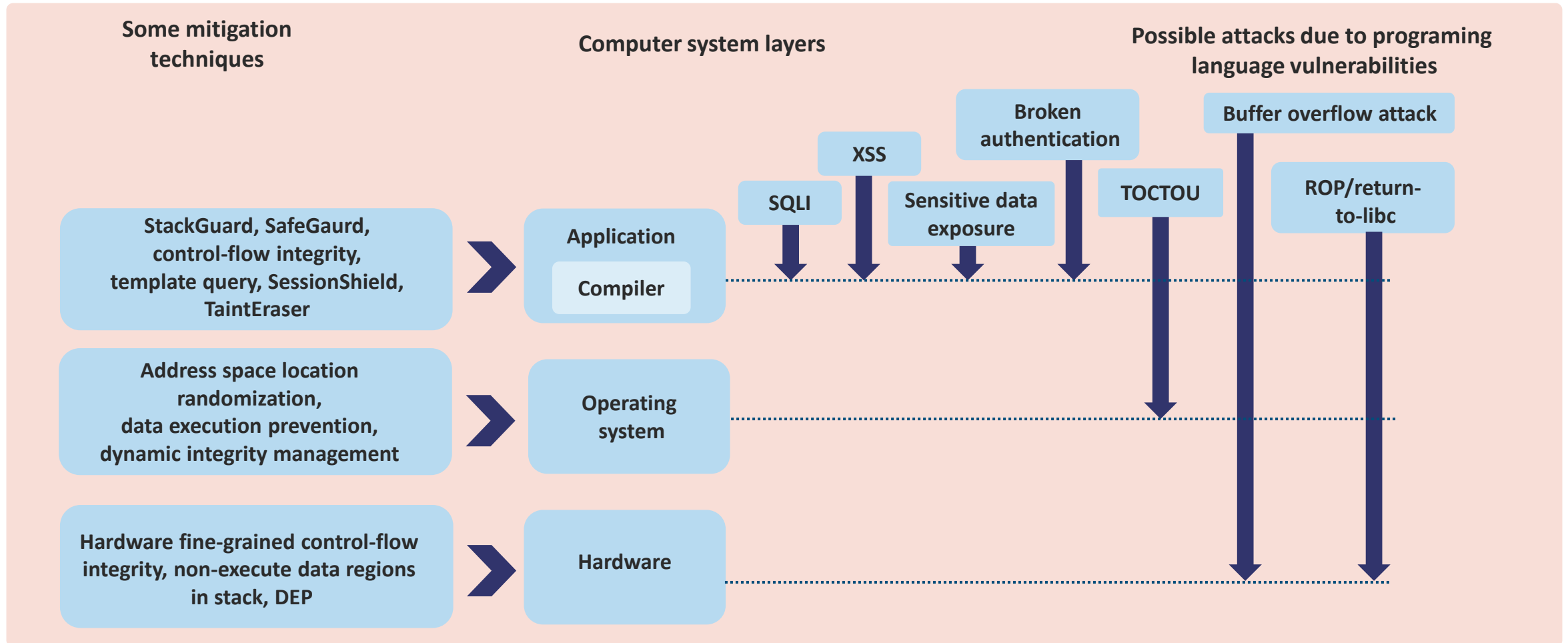


SECURING LANGUAGES, LIBRARIES, AND RUNTIME ENVIRONMENTS

- A critical aspect of DevSecOps and security by design is to secure and harden the programming language, libraries, and the eventual runtime environment
- Most developers and development teams will rely on tools and engines in the integrated platforms to provide a variety of tests, checks, and validation of code
- Teams will also leverage regular peer reviews and automated visibility processes



COMMON PROGRAMMING SECURITY CONTROLS



Khwaja, Amir A., Muniba Murtaza, and Hafiz F. Ahmed. "A Security Feature Framework for Programming Languages to Minimize Application Layer Vulnerabilities." Wiley Online Library. John Wiley & Sons, Ltd, November 7, 2019. <https://onlinelibrary.wiley.com/doi/full/10.1002/spy2.95>.

INTEGRATED DEVELOPMENT ENVIRONMENTS (IDEs)

- **AWS Cloud9** is a cloud-based integrated development environment (IDE) that enables DevOps professionals to construct, run, and debug the code with only a browser
- It includes a code editor, debugger, and terminal
- Cloud9 comes prepackaged with vital tools for popular programming languages, including JavaScript, Python, and PHP
- There is no need to install files or configure the development machine for new projects

Amazon Cloud9

The screenshot displays the Amazon Cloud9 IDE interface. The main editor shows a JavaScript file named `index.js` with the following code:

```
90 };
91
92 const handlersp = {
93   'LaunchRequest': function () {
94     this.emit('GetFact');
95   },
96   'GetNewFactIntent': function () {
97     this.emit('GetFact');
98   },
99
100 },
101 'GetFact': function () {
102   // Get a random space fact from the space facts list
103   // Use this.t() to get corresponding language data
104   const factArr = this.t('FACTS');
105   const factIndex = Math.floor(Math.random() * factArr.length);
106   const randomFact = factArr[factIndex];
107
108   // Create speech output
109   const speechOutput = this.t('GET_FACT_MESSAGE') + randomFact;
110   this.emit(':tellWithCard', speechOutput, this.t('SKILL_NAME'),
111 },
112 'AMAZON.HelpIntent': function () {
113   const speechOutput = this.t('HELP_MESSAGE');
114   const reprompt = this.t('HELP_MESSAGE');
115   this.emit(':ask', speechOutput, reprompt);
116 },
117
```

On the right side, the **Environment Members** panel shows three members: **You (online)**, **aaron (online)**, and **rob (online)**, each with a **RW** (Read/Write) permission. Below this is the **Group Chat** panel, which contains a chat history and a message input field. The chat history shows messages from **You**, **aaron**, and **rob**. The **AWS Resources** and **Debugger** panels are also visible on the right.

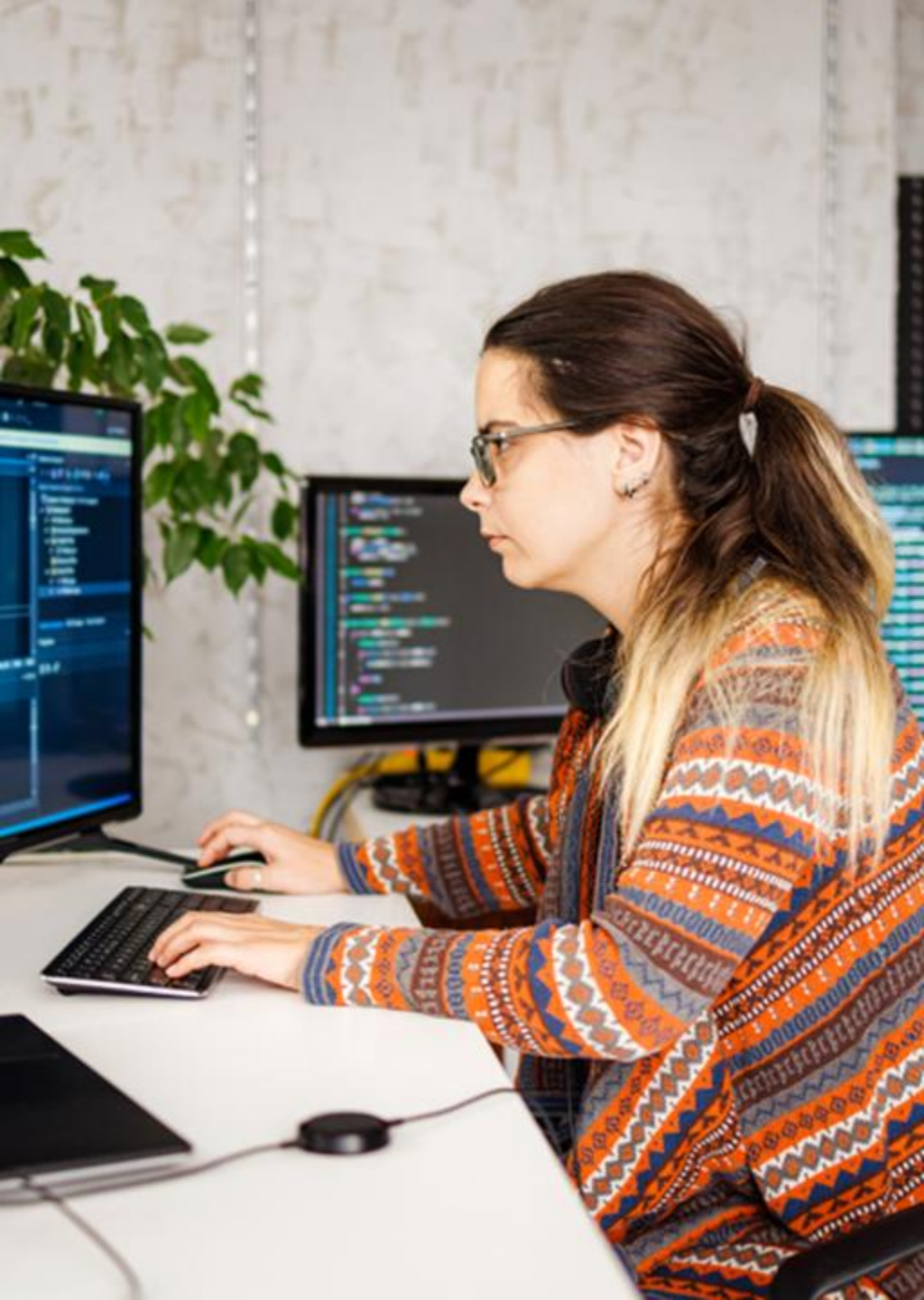
Three callouts (A, B, and C) are present, pointing to specific elements in the interface:

- Callout A** points to the **Environment Members** panel.
- Callout B** points to the **Group Chat** panel.
- Callout C** points to the **Message Input** field in the chat panel.

CONTINUOUS INTEGRATION AND CONTINUOUS DELIVERY (CI/CD)

- CI/CD is viewed as a DevSecOps-enabling practice that supports secure software development by:
 - Automating build, test, and deployment pipelines
 - Reducing human error and manual intervention
 - Enabling rapid iteration with embedded security controls

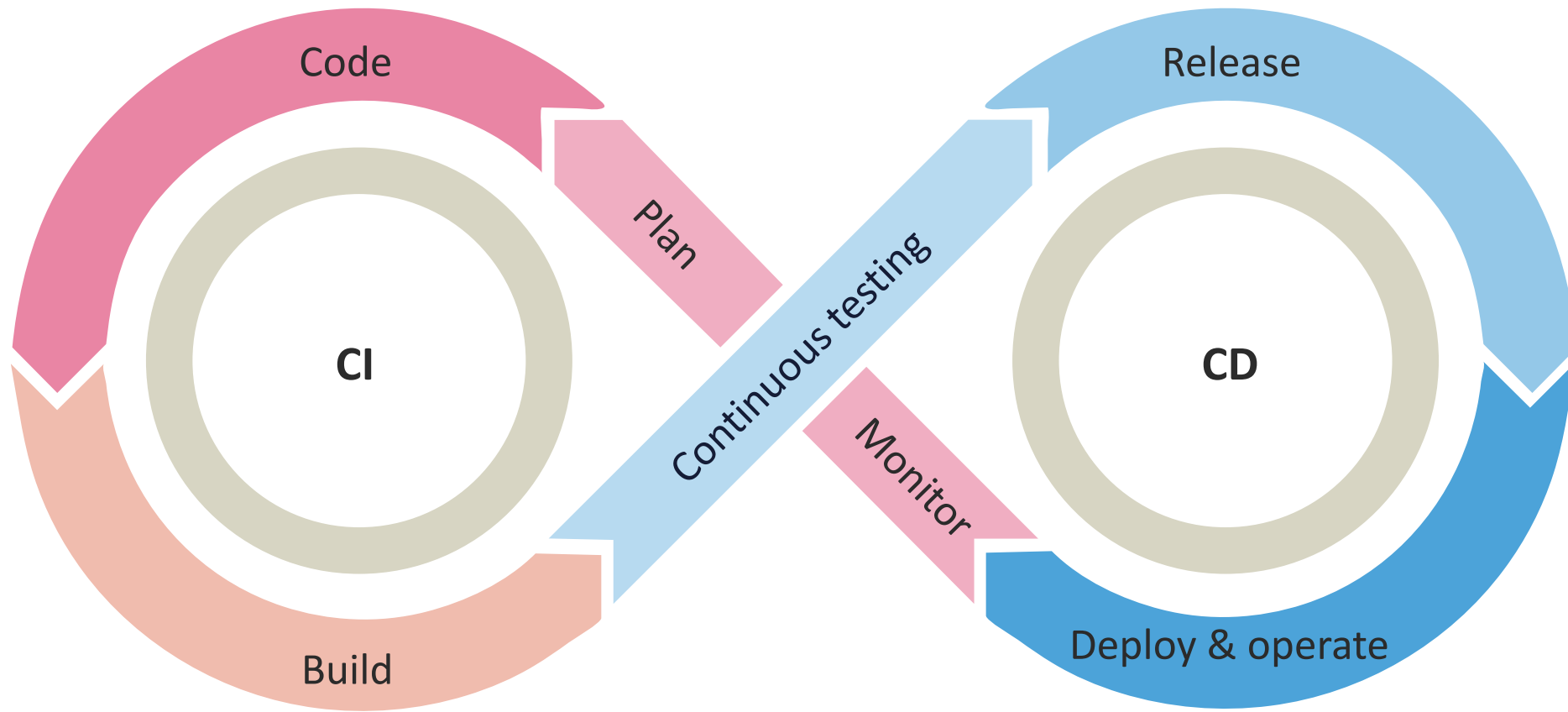




CI/CD ADVANTAGES

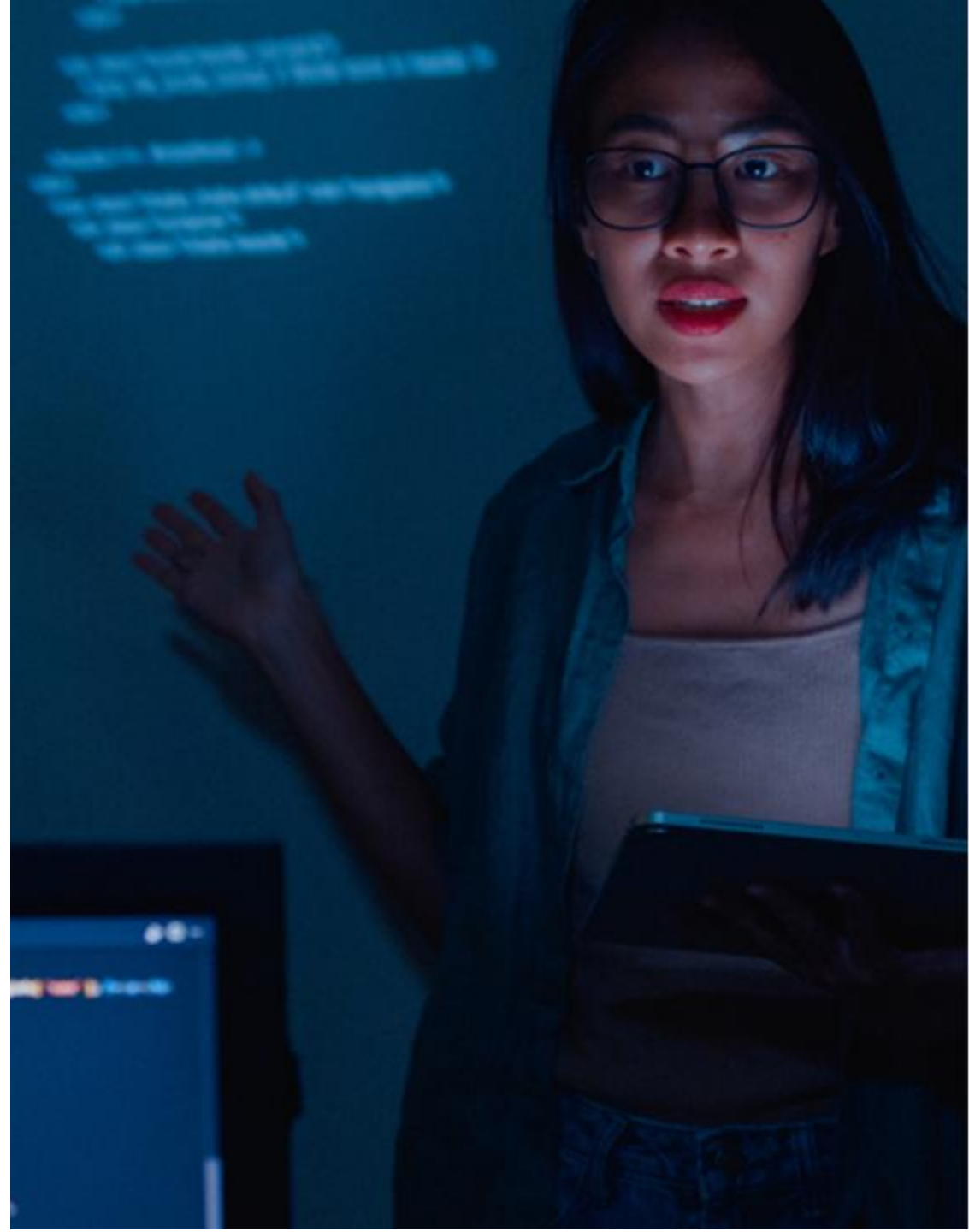
- **Early Detection of Vulnerabilities** - Static and dynamic code analysis tools can be embedded in the pipeline to catch flaws before production
- **Faster Recovery and Rollback** - Automated deployment allows for quick rollback in case of security incidents or failed releases
- **Improved Consistency and Auditability** - Version control and automated logging support traceability and compliance
- **Security as Code** - Infrastructure and security policies can be codified, versioned, and tested like application code

CI/CD PIPELINE



SOFTWARE CONFIGURATION MANAGEMENT (SCM)

- Software configuration management (SCM) is a software engineering process to systematically manage, organize, and control the changes in the documents, codes, and other artifacts during the SDLC
- SCM is part of the cross-disciplinary field of configuration management (integrated product teams) and can correctly determine the revision history





REASONS TO USE SCM

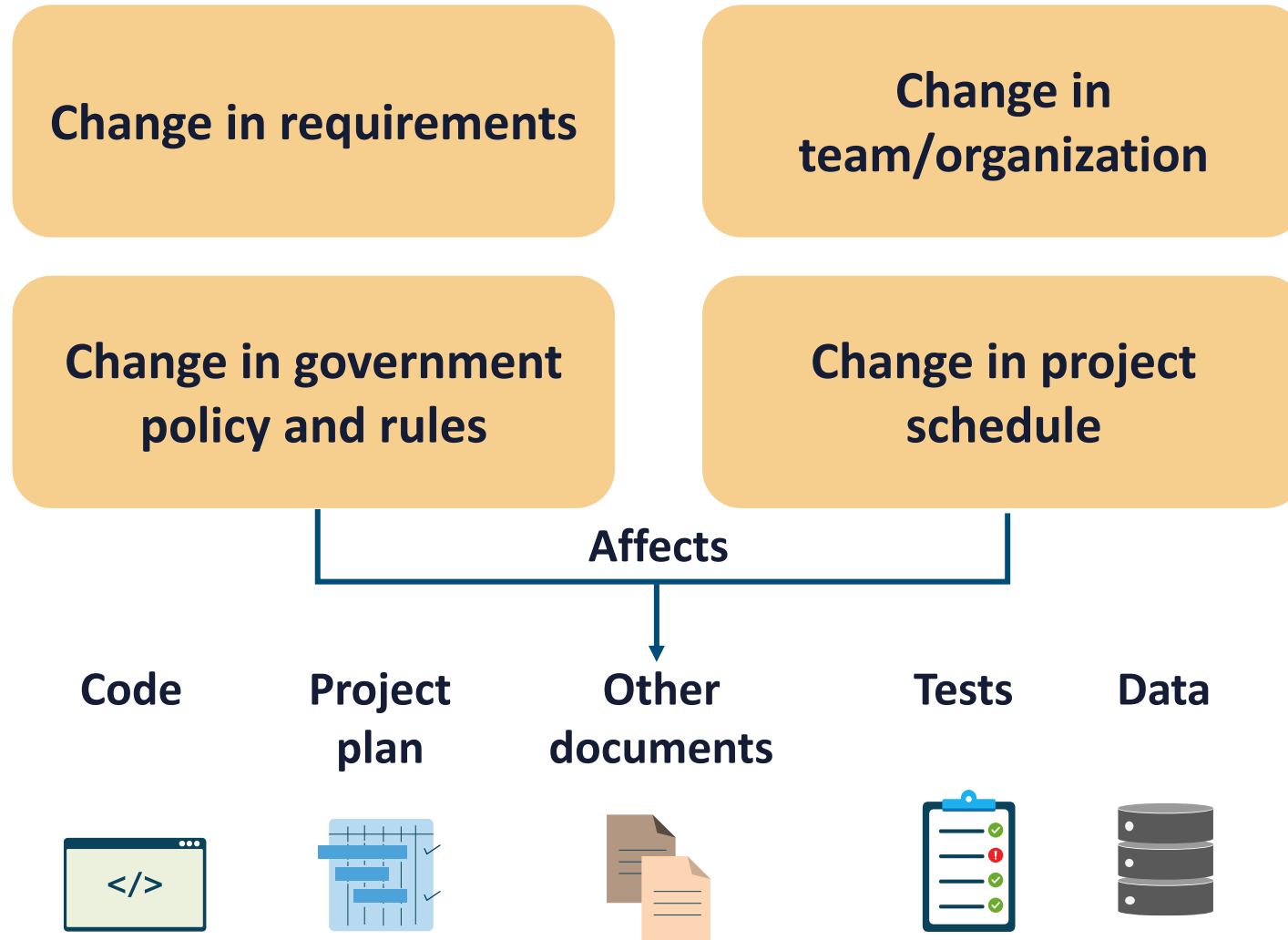
- To enhance productivity and minimize errors
- When there are several people working on applications that are continually updating (CI/CD or spiral development)
- If there are multiple versions, branches, microservices, and programmers involved in a software project, and the team is geographically dispersed yet is working concurrently



REASONS TO USE SCM

- Changes in customer requirements, policy, budget, and schedules need to be accommodated
- Software must be able to run on different platforms and operating systems
- There is a critical need to develop coordination among cross-functional stakeholders
- The costs involved in making changes to an app need to be controlled

SOFTWARE CONFIGURATION MANAGEMENT (SCM)



CODE REPOSITORY SECURITY

- Code is only as secure as the methods and systems used to generate it
- **Some of the advantages that come with using a secure code repository are version control, peer review, and built-in auditing**
- It is critical that the repository (such as GitHub or Azure) is an adequately secure central point of code storage and management

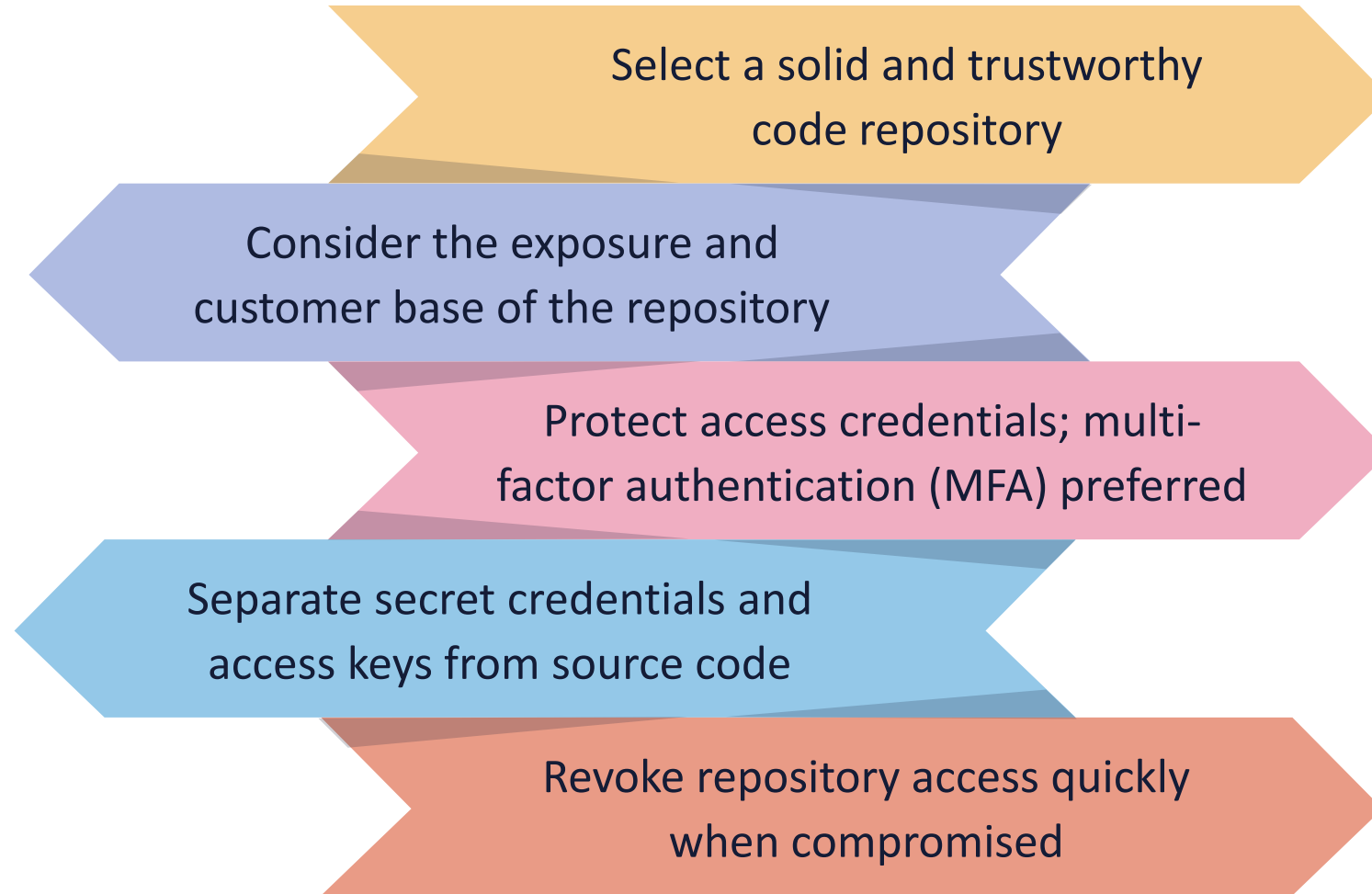




CODE REPOSITORY SECURITY

- Attackers can change a code base without your knowledge or permission due to loss/compromise of access credentials or breach of the core service
- If appropriate due diligence is applied to security measures, the benefits of using a code repository far outweigh the risks

CODE REPOSITORY SECURITY



STATIC APPLICATION SECURITY TESTING (SAST)

- SAST tools are also known as code analyzers that conduct a direct white-box analysis of the application source code
- The analysis runs on a static view of code, in that the code is not running at the time of the assessment





SAST

- SAST security tools are mainstream and are widely adopted throughout the software industry
- They have broad programming language support and use concepts that are relatively easy to comprehend
- SAST code analyzers have no visibility of the execution flow, can be slow, inaccurate, and outdated, and often need additional customization and/or tuning



DYNAMIC APPLICATION SECURITY TESTING (DAST)

- DAST tools are most often web scanners like OWASP ZAP and Burp Suite (vulnerability scanners)
- When compared to SAST, they perform black-box analysis in that they do not have access to the code or the implementation specifics
- DAST only inspects the system's responses to a series of tests designed to highlight vulnerabilities

DAST

- DAST tools function independently of the underlying application platform and offer solid support for manual penetration testing
- A top-level DAST detects only ~20% of issues with no information provided on the location of the issue in the code base
- An experienced security background is necessary to interpret the results





INTERACTIVE APPLICATION SECURITY TESTING (IAST)

- IAST combines the advantages of SAST and DAST solutions
- It offers the benefits of a static view, because it can see the source code
- It also delivers the aids of a web scanner approach, since it sees the execution flow of the application during runtime
- It reportedly can detect ~100% of OWASP benchmarks in real-time with no false positives

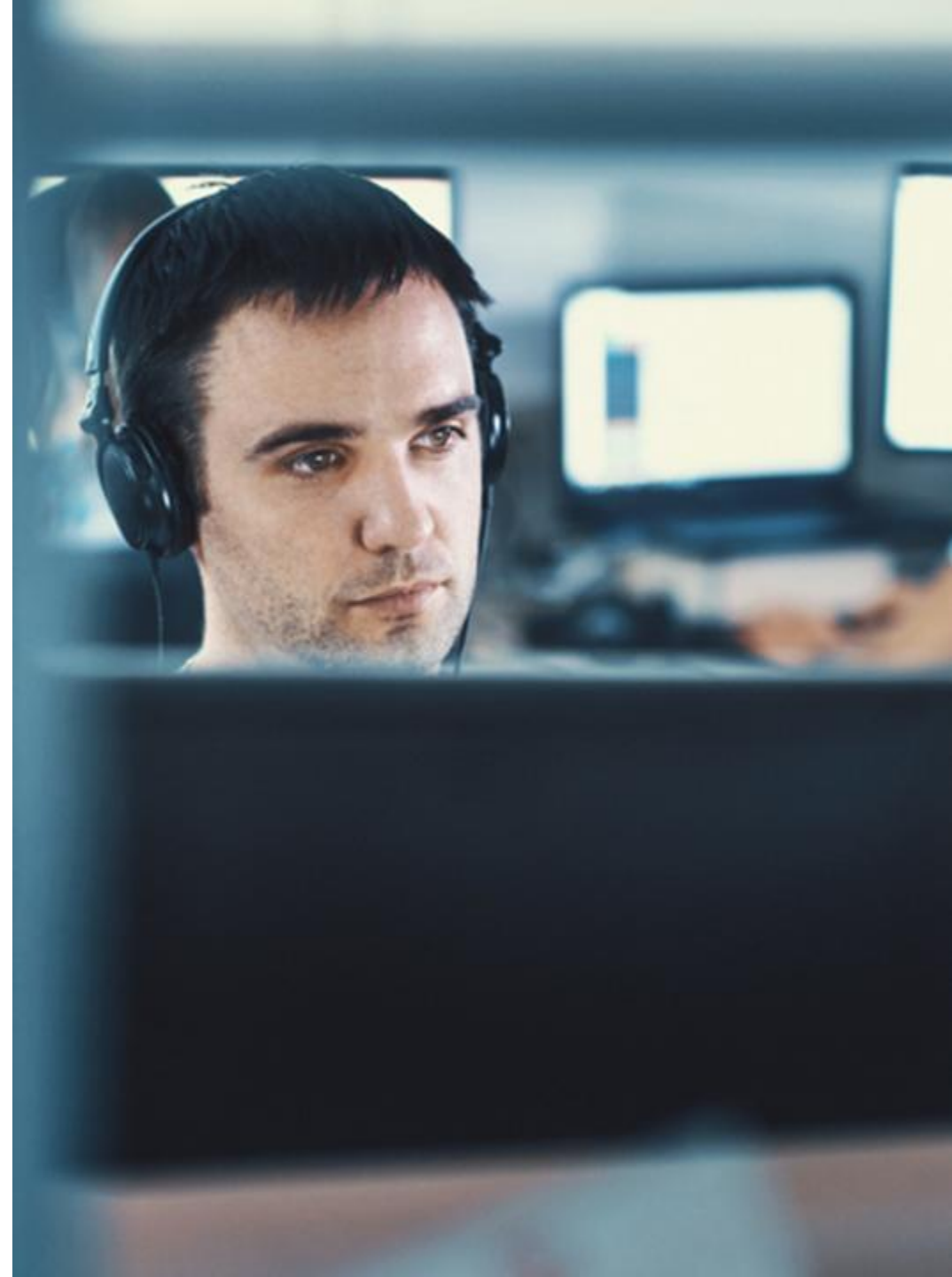


IAST

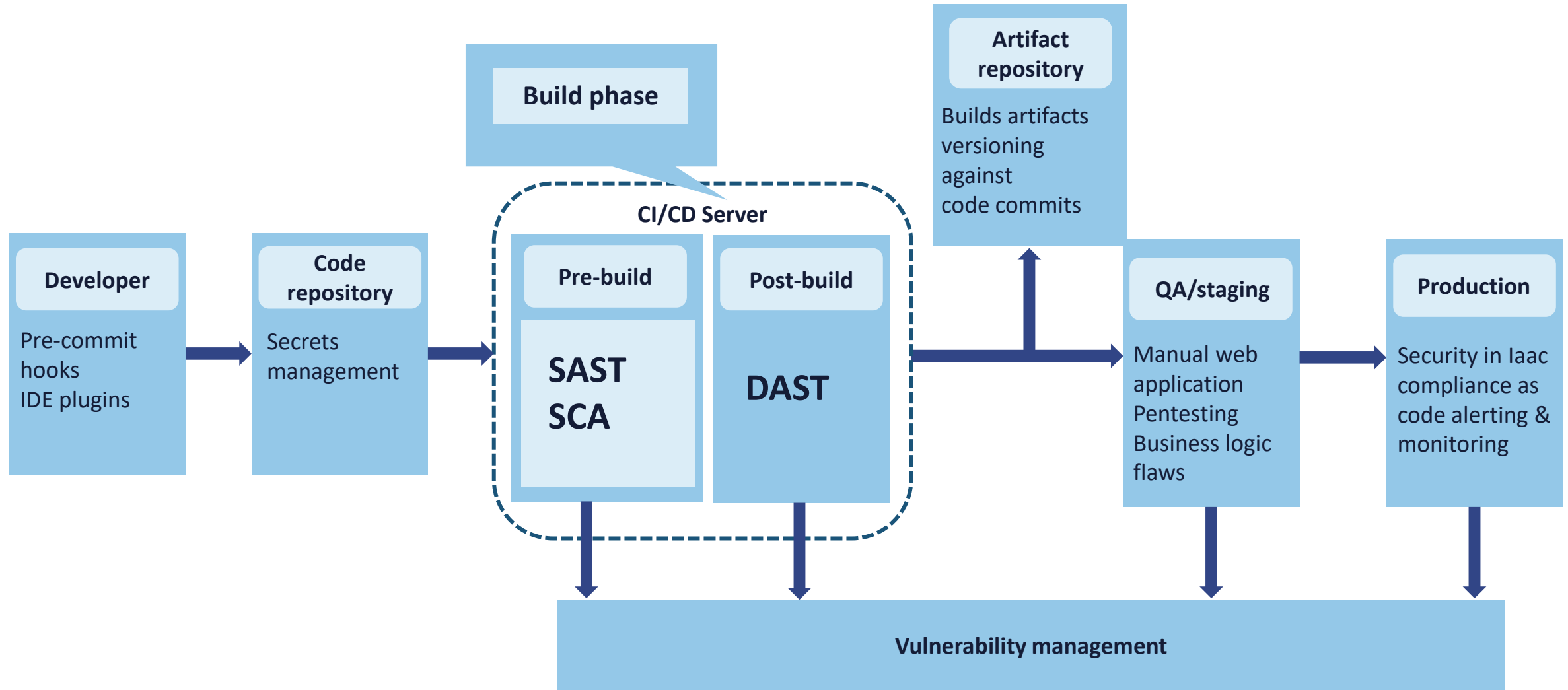
- IAST can flexibly be used in quality assurance (QA) and production environments, analyzing dependencies as well as legacy components
- There is no need to scan or attack the application
- IAST does continuous detection
- It is DevSecOps-friendly
- IAST integrates and communicates with task management systems to create unified workflows

SOFTWARE COMPOSITION ANALYSIS (SCA)

- SCA is a development technique for recognizing and controlling the open-source and third-party aspects of enterprise applications
- It assists teams in detecting app security vulnerabilities, licensing problems, or obsolete software components
- SCA also offers a software bill of materials (SBOM) to generate a comprehensive inventory of the open-source software packages and their roots



SOFTWARE COMPOSITION ANALYSIS



ASSESSING SOFTWARE SECURITY AND CODING GUIDELINES

Objectives

- Assess auditing, change logs, and risk analysis and mitigation
- Assess acquired software, managed services, and cloud services
- Describe source code vulnerabilities and API weaknesses in greater detail
- Outline secure coding practices
- Provide an overview of software-defined security (SDS)



ASSESSING AUDITING & LOGGING OF CHANGES

- Although the process of auditing produces a wide variety of results and supports accreditation and certification, the internal and external auditing tasks themselves must periodically be assessed
- A practical way to assess security auditors is to rely on continual third-party evaluations and certifications such as the Cybersecurity and Infrastructure Security Agency (CISA), Certified Internet Audit Professional (CIAP), ISACA Certified Information Security Manager (CISM), and Cloud Security Alliance (CSA) Certificate of Cloud Auditing Knowledge (CCAK)

ASSESSING AUDITING & LOGGING OF CHANGES

- When security information and event management (SIEM) and security orchestration, automation, and response (SOAR) systems are being deployed for centralized logging and automation, they must be fine-tuned for false positives and false negatives on a regular basis
- Auditors must go through regular training programs, conferences, seminars, and more for continuing education and advancement
- Security auditing is an art and a science where the practitioner only gets better with experience and improvement with tools



ASSESSING RISK ANALYSIS AND MITIGATION



- Performing risk assessment and analysis will never be a one-time program or initiative
- Many organizations will rely on ISO, ISACA, OWASP, ITIL 4, MITRE, and other architectures and frameworks to continually assess the success of risk management decisions
- These assessments may be conducted by steering committees, boards, or external accreditors and certifying bodies

GAP ANALYSIS

- A gap is the difference between the implemented existing controls and the predetermined control objectives
- Gap analysis is the outcome of corporate security strategy and governance
- **It is foundational for assessing the success of risk analysis and security control implementation**
- It provides the fundamental information security initiative action plans and programs



GAP ANALYSIS

- Current countermeasures should be established according to the organization's risk appetite for each asset class
- Gap analysis should be performed in a reusable and repeatable manner to assess and report on the efficacy of executed controls
- Focus will be on established metrics such as key performance indicators (KPIs) and key goal indicators (KGIs)



ASSESSING ACQUIRED SOFTWARE: COTS

- Commercial-off-the-shelf (COTS) is a common pre-packaged (with varying licenses) and as-is solution
- COTS products are intended to be easily installed and to interoperate tightly with existing system components
- Almost all software bought by the public computer user fits into the COTS category (operating systems, office product suites, word processing, and email programs)



ASSESSING ACQUIRED SOFTWARE: MOTS



- MOTS stands for either modified or modifiable off-the-shelf software
- It is typically a product whose source code can be modified or customized by the purchaser, the vendor, or another party to meet the needs of the customer
- The evolution of Agile and CI/CD rapid development of containerized applications and microservices has led to more MOTS solutions
- **Modifications may be driven by the results of ongoing auditing and assessment of the acquired software solutions**

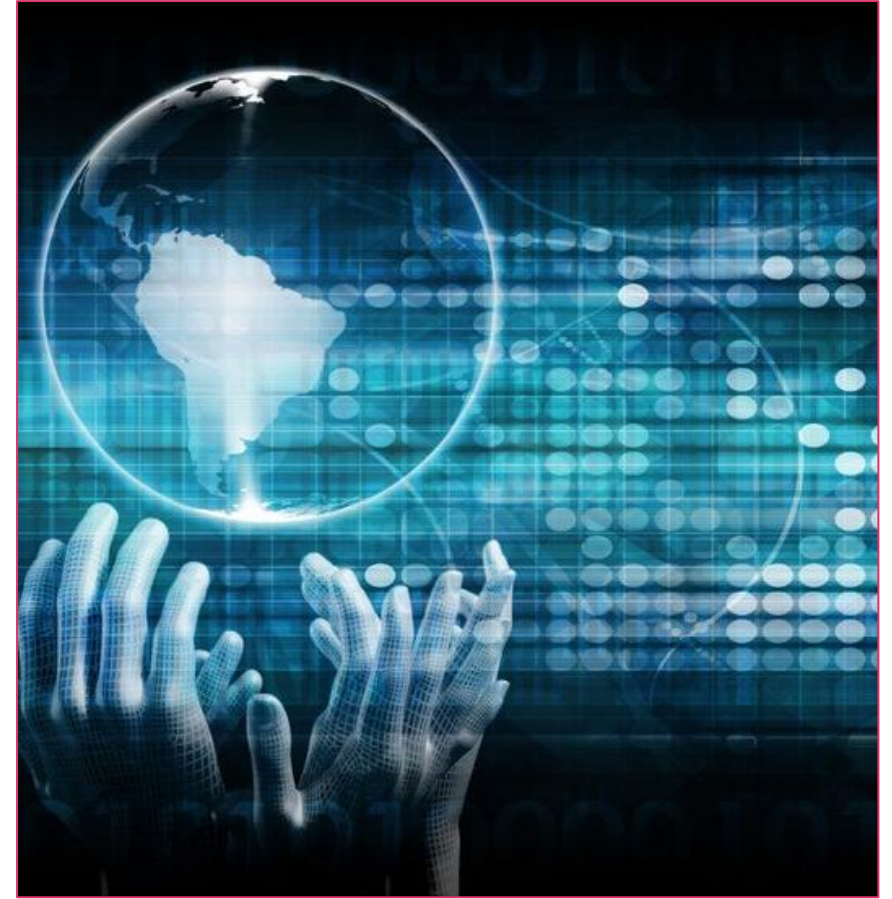
A man and a woman are looking at a computer screen. The man is in the foreground, wearing glasses and a maroon shirt. The woman is behind him, also wearing glasses and a striped shirt. The screen displays various digital elements like bar charts, pie charts, and lines of code. A pink diagonal bar is on the right side of the image.

ASSESSING OPEN-SOURCE SOFTWARE SOLUTIONS

- Open-source software is a package whose code is accessible for public examination, alteration, and improvement
- **Many enterprises and products (90% by some estimates) use at least one open-source component, often without being aware of it**
- Normally, this software is built using public community collaboration and is preserved and updated on a voluntary basis
- Open-source software can be used according to a diversity of licenses, depending on what the developers have implemented

ASSESSING RISK ANALYSIS AND MITIGATION

- It is critical to evaluate and assess all open-source solutions because
 - Vulnerabilities are publicly known
 - There are no claims or obligations to be secure
 - Open-source software often includes or demands the use of vulnerable third-party libraries
 - There may be intellectual property challenges
 - There are over 200 types of licenses that can be used with open-source software
 - There is a general lack of warranty for its security, support, or content





ASSESSING MANAGED SERVICES

- Many enterprise solutions today are vendor-based or cloud-based Platform as a Service (PaaS) or Software as a Service (SaaS) solutions
- The key factor with these services is to perform proper due diligence and due care in formulating service-level agreements, master service agreements, and reciprocal agreements

ASSESSING MANAGED SERVICES

- There should be a regular review of security policies for remote access and single sign-on, federated access, and least privilege when accessing managed services with providers
- The CSA offers a Security Trust Assurance and Risk (STAR) registry with questionnaires filled out by many providers based on the Cloud Controls Matrix (CCM)
- The CCM is 197 control objectives structured in 17 domains covering all key aspects of cloud technology



ASSESS CLOUD SERVICES WITH THE CCM

- It can be used to audit and evaluate every participant and agent within the cloud supply chain
- It is considered the de facto standard for cloud security assurance and compliance
- The STAR Level 1: Security Questionnaire (CAIQ v4) offers an industry-accepted way to document what security controls exist in Infrastructure as a Service (IaaS), PaaS, and SaaS services





SOURCE CODE SECURITY WEAKNESSES AND VULNERABILITIES

- Code vulnerabilities are flaws in source code that, when exploited by attackers, will result in data leakage, code tampering, or possibly total data loss
- There are several well-established vulnerability databases and vendor support engines that keep programmers and development teams current on common weaknesses and attacks



SOURCE CODE SECURITY VULNERABILITIES

- Utilizing **hard-coded (embedded) credentials/keys** is considered an unsecured coding practice and can lead to critical issues
- For several reasons, coders tend to still use **weak encryption algorithms and cryptographic hashes** in their solutions, often for performance reasons
- Using standard **pseudo-random number generators** leaves one vulnerable to cryptographic attacks:
 - These leave software prone to insecure randomness errors, which happen when a function producing predictable values is used as a source of randomness

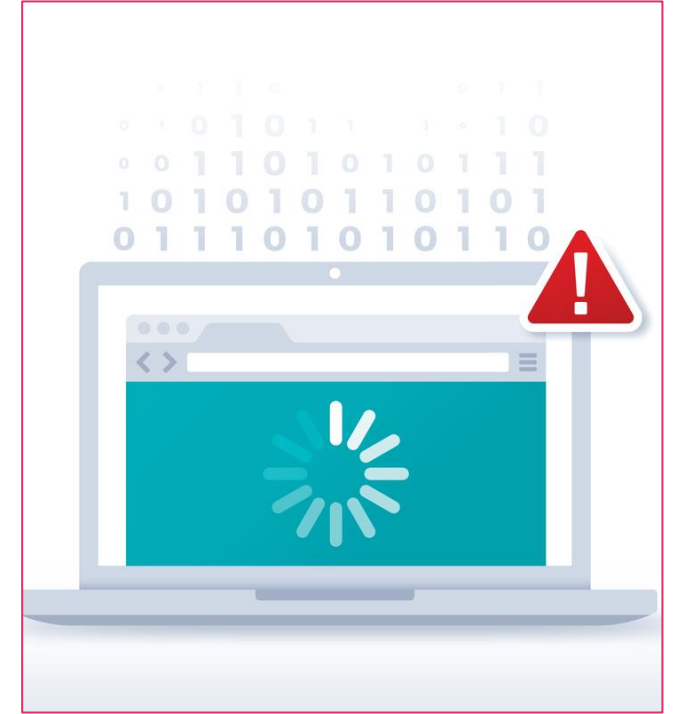
ASSESSING API SECURITY WITH OWASP TOP 10 2023

- **API1:2023 – Broken Object Level Authorization:**
 - APIs tend to expose endpoints that handle object identifiers so object-level authorization checks should be considered in every function that accesses a data source using an ID from the user
- **API2:2023 – Broken Authentication:**
 - Authentication methods are often deployed incorrectly, permitting attackers to compromise authentication tokens or exploit implementation flaws
- **API3:2023 – Broken Object Property Level Authorization:**
 - A lack of or improper authorization validation at the object property level can result in data exposure or manipulation by unauthorized parties



ASSESSING API SECURITY WITH OWASP TOP 10 2023

- **API4:2023 – Unrestricted Resource Consumption:**
 - Meeting API requests demands resources such as network bandwidth, CPU, memory, and storage
- **API5:2023 – Broken Function Level Authorization:**
 - Authorization flaws often occur with intricate access control policies that involve different hierarchies, groups, and roles
 - There can also be an unclear separation between administrative and normal functions
- **API6:2023 – Unrestricted Access to Sensitive Business Flows:**
 - APIs may expose data flows such as buying a ticket or posting a comment, without considering how the functionality might damage the business if done excessively in an automated way



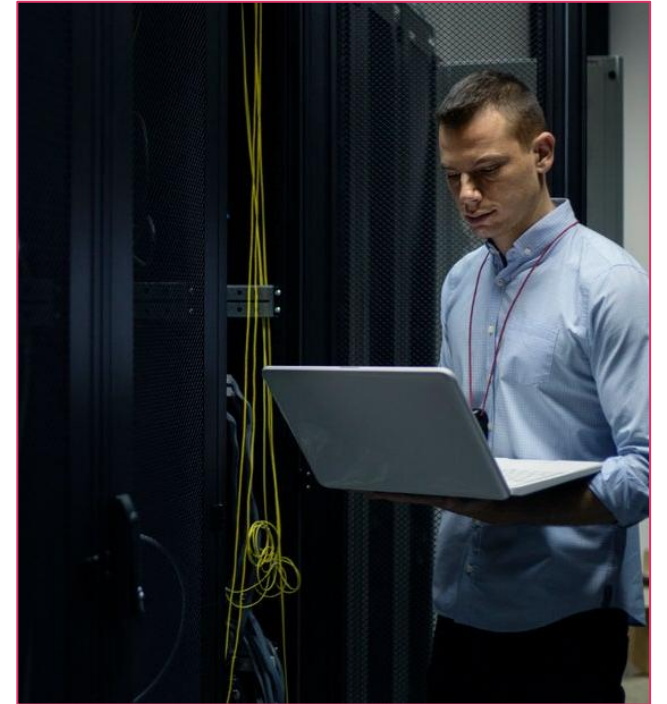


ASSESSING API SECURITY WITH OWASP TOP 10 2023

- **API7:2023 – Server-Side Request Forgery:**
 - A Server-Side Request Forgery (SSRF) can happen when an API is fetching a remote resource without validating the user-supplied URI
 - This lets an attacker trick the application into sending a fashioned request to an unexpected target, even through a firewall or a VPN
- **API8:2023 – Security Misconfiguration:**
 - APIs and the systems that support them usually have complicated configurations, intended to make the APIs more customizable

ASSESSING API SECURITY WITH OWASP TOP 10 2023

- **API9:2023 – Improper Inventory Management:**
 - APIs may expose more endpoints than legacy web applications
 - This fact makes proper and updated documentation extremely vital
 - The complete inventory of hosts and deployed API versions is important to alleviate issues like deprecated API versions and exposed debug endpoints
- **API10:2023 – Unsafe Consumption of APIs:**
 - Developers tend to trust data received from third-party APIs more than input from users
 - This can lead to using weaker security standards
 - To compromise APIs, attackers focus on integrated third-party services instead of directly attacking the target API



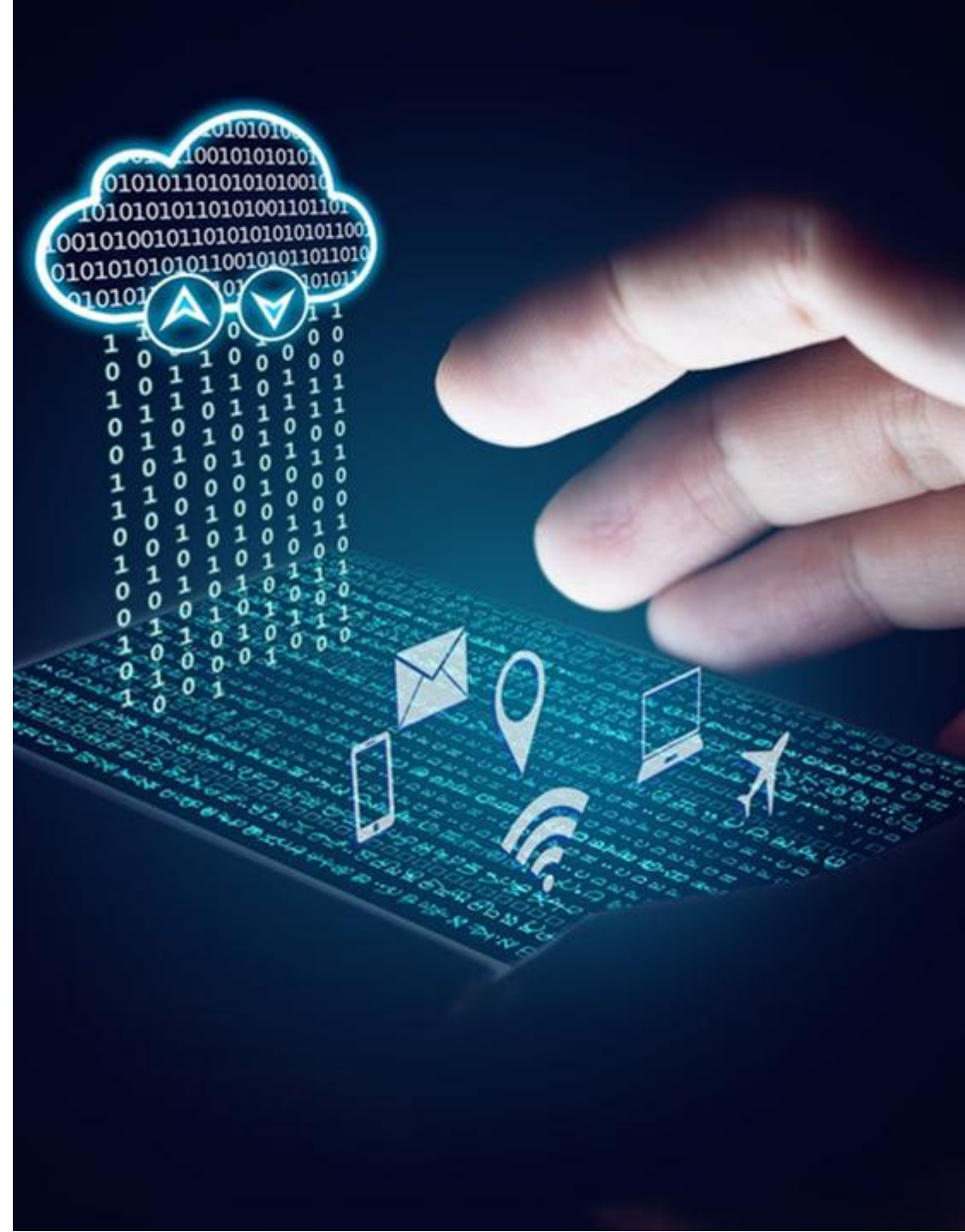


OWASP ENTERPRISE SECURITY API (ESAPI) PROJECT

- ESAPI is a free and open-source web application security control library that empowers programmers to construct lower-risk applications and interfaces
- The ESAPI libraries are intended to make it stress-free for developers to retrofit security into existing applications
- It offers a collection of security control interfaces that define various types of parameters passed to security controls

OWASP APPLICATION SECURITY VERIFICATION STANDARD (ASVS)

- The ASVS project gives developers a list of requirements for secure development
- The standard offers a basis for testing application technical security controls, as well as any technical security controls in the environment, that are used to mitigate attacks, such as XSS and SQL injection
- The requirements were developed to be used:
 - As a meaningful metric
 - As secure development guidance
 - During the procurement process



SOFTWARE-DEFINED SECURITY (SDS)

- SDS is a model in which information security is highly controlled, often using virtualization
- The functionality of network security devices, such as next-gen firewalls, intrusion detection and prevention, identity and access controls, and network segmentation, are removed from hardware devices to a software layer
- SDS exploits the software-defined networking (SDN) initiative to enhance network security



SDS ADVANTAGES

- Offers resourceful and dynamic countermeasures to security attacks
- Separates security away from traditional hardware vulnerabilities
- Ability to dynamically configure existing network nodes allows for rapid attack mitigation from zero-day attacks
- Provides a synchronized view of logical security policies within the SDN controller model (not tied to any server or specialized security device)





SDS ADVANTAGES

- Provides visibility of information from one source
- Integrates with emerging technology to correlate events in a simpler way and respond more efficiently and intelligently to threats
- Enables centralized management of security, which is implemented, controlled, and managed by security software through the SDN controller
- Facilitates Internet of Things (IoT) and Bring Your Own Device (BYOD) connectivity and security

SDS FRAMEWORK

